

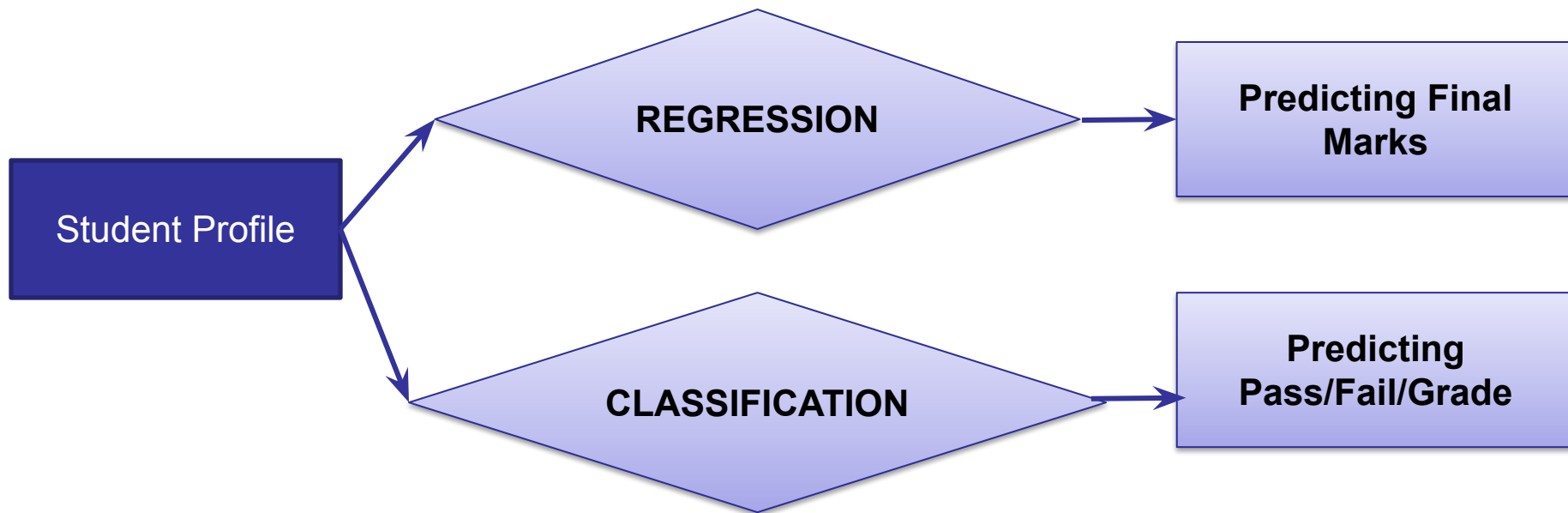
CSE 445: Machine Learning

Classification



Supervised Learning

- Regression □ involves predicting values (continuous)
- Classification □ predicting classes (discrete)



MNIST

- 70,000 small images of handwritten images
 - The “Hello, World” of Machine Learning
 - Each image has 28 x 28 pixels, which comes out to 784 features
 - Each feature represents a pixel's intensity from 0 to 255
 - Comes pre-shuffled, with the first 60,000 images for training, and the remaining 10,000 images for testing

```
>>> X, y = mnist["data"], mnist["target"]
>>> X.shape
(70000, 784)
>>> y.shape
(70000,)

import matplotlib as mpl
import matplotlib.pyplot as plt

some_digit = X[0]
some_digit_image = some_digit.reshape(28, 28)

plt.imshow(some_digit_image, cmap="binary")
plt.axis("off")
plt.show()
```



Digits from MNIST dataset



Figure 3-1. Digits from the MNIST dataset

Classification Performance Measure

- Classification often deals with skewed datasets
 - Accuracy is not the preferred performance measure!
- e.g. even an untrained binary classifier can have 90% accuracy if one of the classes constitute just 10% of images!

```
>>> from sklearn.model_selection import cross_val_score
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.96355, 0.93795, 0.95615])
```

```
from sklearn.base import BaseEstimator

class Never5Classifier(BaseEstimator):
    def fit(self, X, y=None):
        return self
    def predict(self, X):
        return np.zeros((len(X), 1), dtype=bool)
```

```
>>> never_5_clf = Never5Classifier()
>>> cross_val_score(never_5_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.91125, 0.90855, 0.90915])
```

Confusion Matrix

- Count the number of times class A is correctly classified, and the number of times it's misclassified as a separate class
 - How often does the classifier “confuse” handwritten 5 with 3?

Actual \ Predicted	Not 5	5
	Not 5	5
Not 5	53057 (True Negative)	1522 (False Positive)
5	1325 (False Negative)	4096 (True Positive)

$$Precision = \frac{TP}{TP+FP}$$

$$F1 \text{ Score} = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$

$$= \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$Recall = \frac{TP}{TP+FN}$$

Confusion Matrix

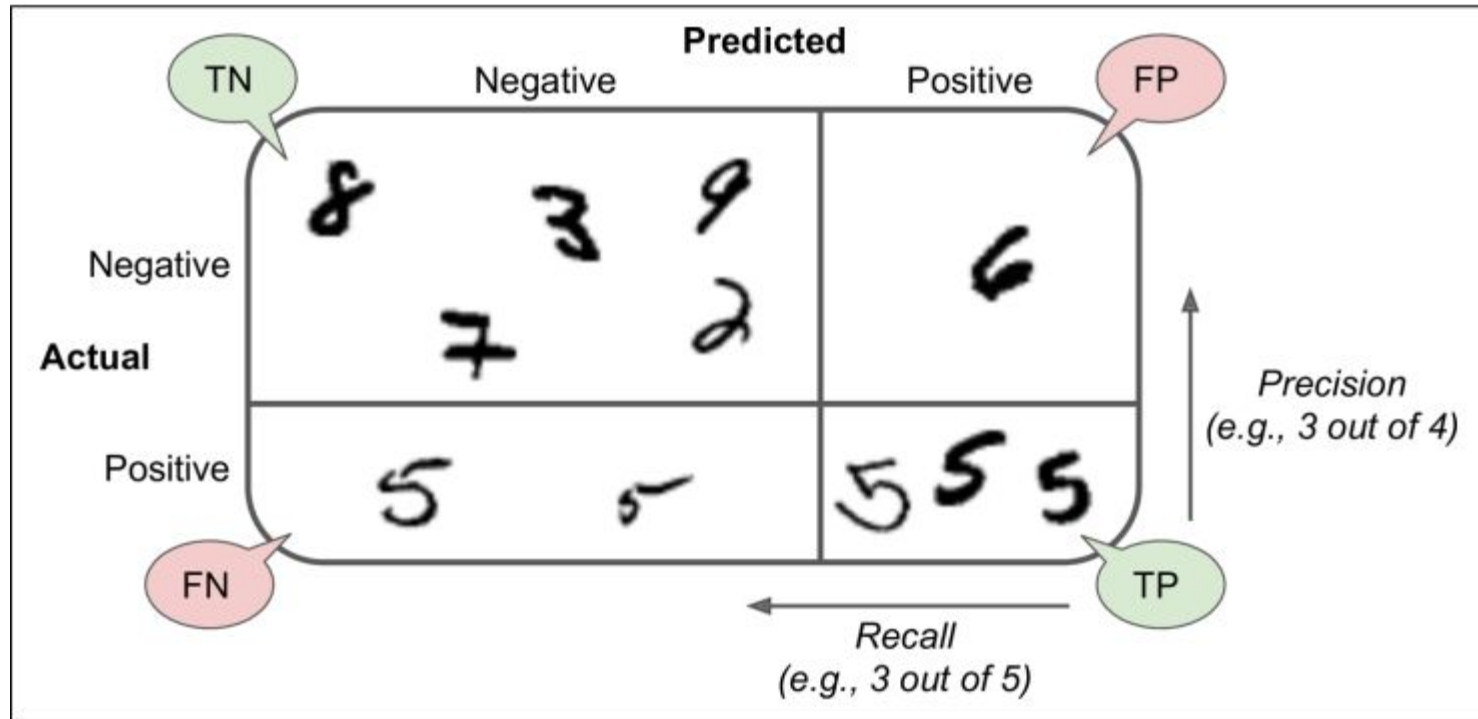


Figure 3-2. An illustrated confusion matrix shows examples of true negatives (top left), false positives (top right), false negatives (lower left), and true positives (lower right)

Precision/Recall Tradeoff

- Increasing classifier precision leads to lower recall, and vice versa!

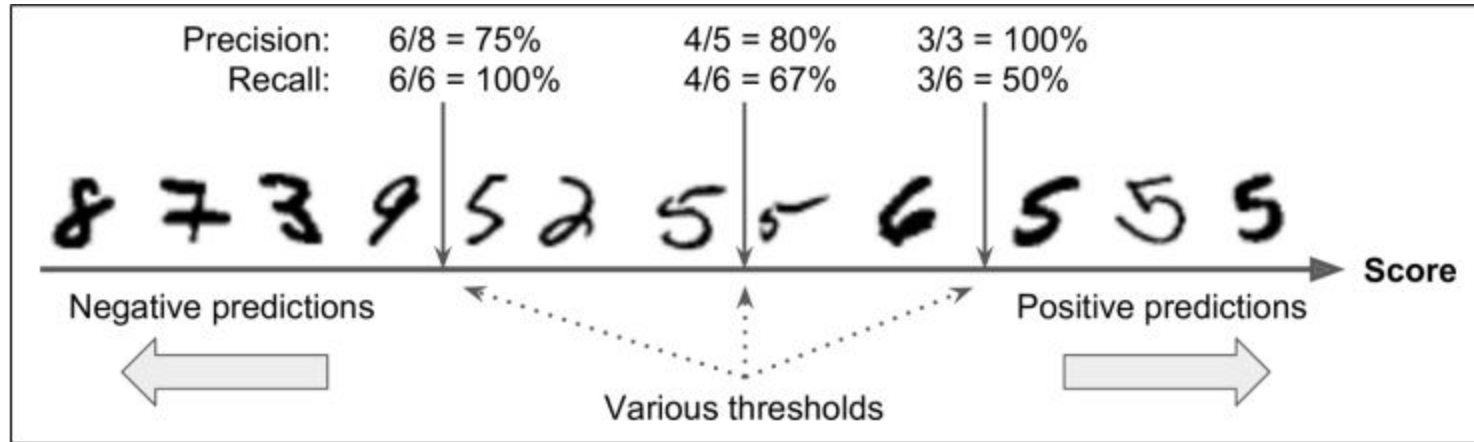
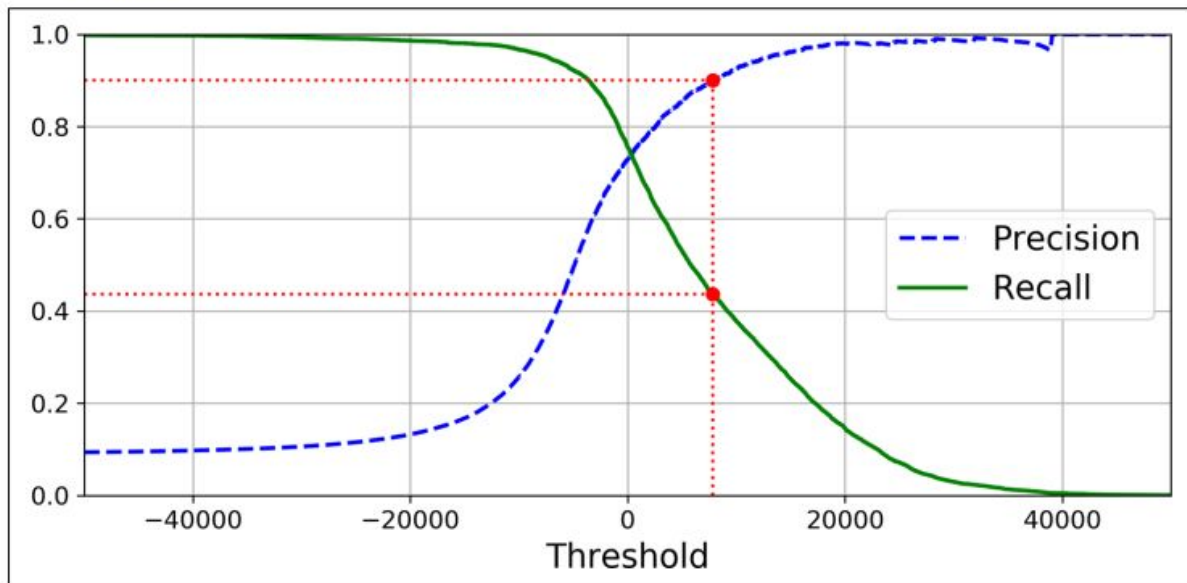


Figure 3-3. In this precision/recall trade-off, images are ranked by their classifier score, and those above the chosen decision threshold are considered positive; the higher the threshold, the lower the recall, but (in general) the higher the precision

Precision/Recall Curve

```
def plot_precision_recall_vs_threshold(precisions, recalls, thresholds):  
    plt.plot(thresholds, precisions[:-1], "b--", label="Precision")  
    plt.plot(thresholds, recalls[:-1], "g-", label="Recall")  
    [...] # highlight the threshold and add the legend, axis label, and grid  
  
plot_precision_recall_vs_threshold(precisions, recalls, thresholds)  
plt.show()
```



Fairly easy to create
a classifier with high
precision – but at
what recall?

Figure 3-4. Precision and recall versus the decision threshold

Receiver Operating Characteristic (ROC) curve

- Plots:
 - True Positive Rate (TPR $\square$ another name for Recall/Sensitivity) against
 - False Positive Rate (FPR $\square$ Ratio of negative instances that are incorrectly classified as positive)
 - $FPR = 1 - TNR = 1 - \text{Specificity}$
- ROC curve plots sensitivity (recall) against (1- specificity)

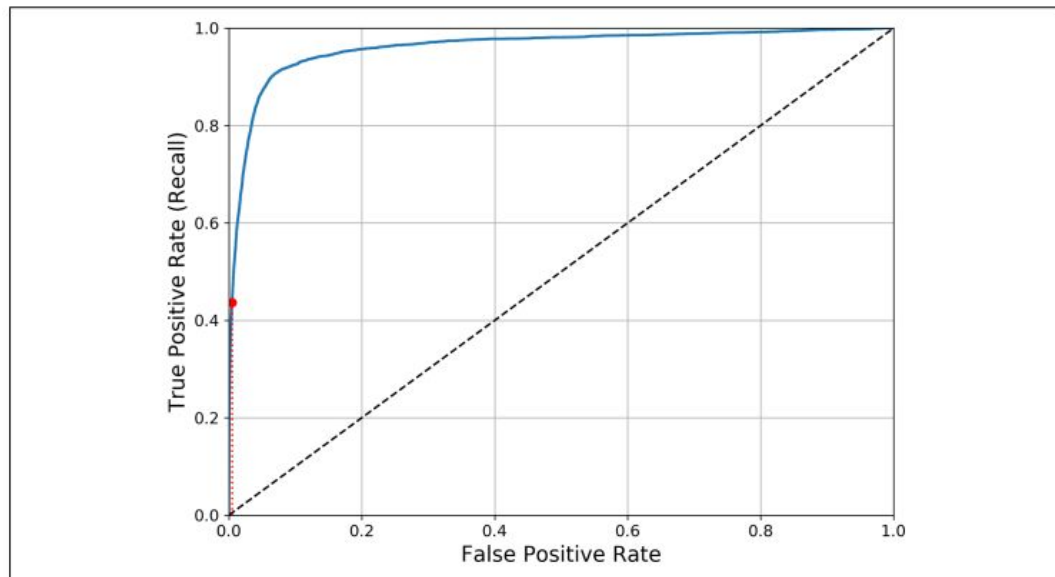


Figure 3-6. This ROC curve plots the false positive rate against the true positive rate for all possible thresholds; the red circle highlights the chosen ratio (at 43.68% recall)

Receiver Operating Characteristic (ROC) curve

- Higher recall/TPR leads to more false positives
- A good classifier stays very close to the top left corner
 - High TPR, and low FPR
- AUC – Area under the curve
 - Perfect classifier will have an AUC equal to 1
 - Purely random classifier (dotted diagonal line) will have an AUC equal to 0.5
- Use PR curve when positive class is rare in dataset, or when false positives are a greater concern than false negatives

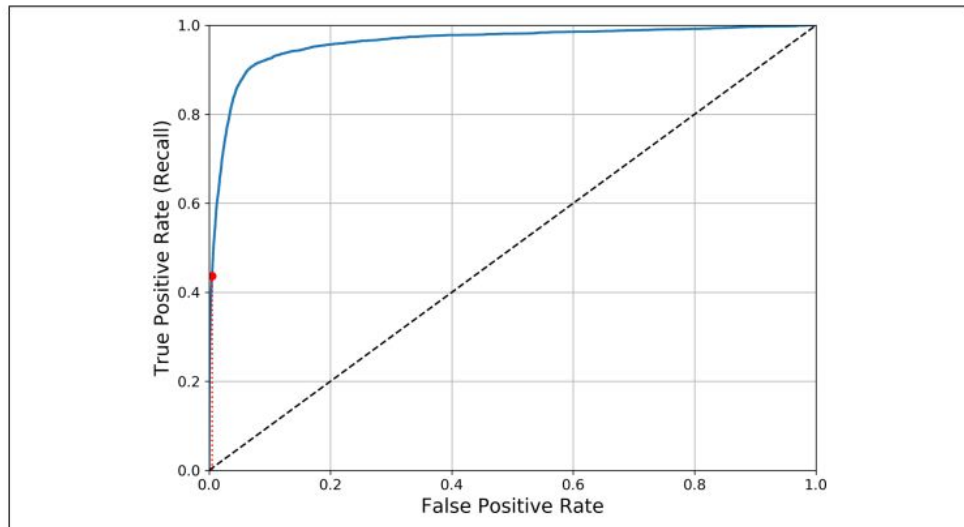


Figure 3-6. This ROC curve plots the false positive rate against the true positive rate for all possible thresholds; the red circle highlights the chosen ratio (at 43.68% recall)

```
>>> from sklearn.metrics import roc_auc_score
>>> roc_auc_score(y_train_5, y_scores)
0.9611778893101814
```

Random Forest Classifier

- Ensemble classification with multiple decision trees
- Better than SGD Classifier as evident in ROC curve
- 99% precision and 86.6% recall makes this a better classifier in those regards as well

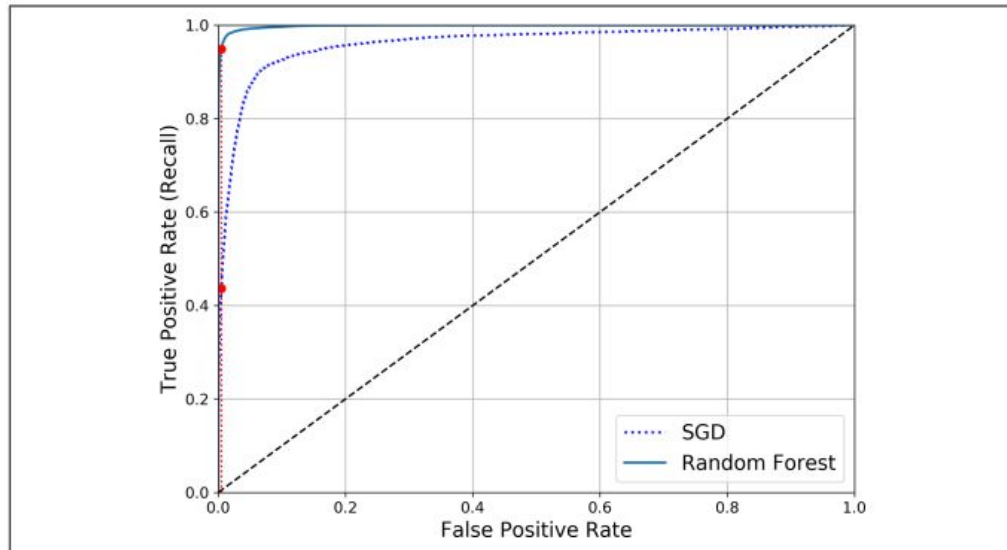


Figure 3-7. Comparing ROC curves: the Random Forest classifier is superior to the SGD classifier because its ROC curve is much closer to the top-left corner, and it has a greater AUC

```
from sklearn.ensemble import RandomForestClassifier
```

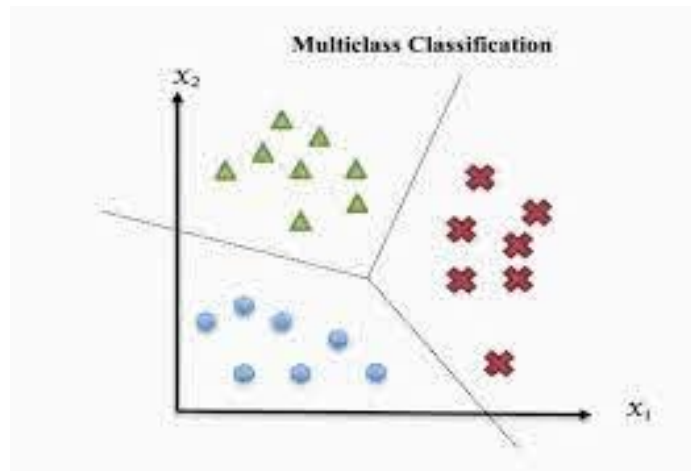
```
forest_clf = RandomForestClassifier(random_state=42)  
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,  
                                   method="predict_proba")
```

```
y_scores_forest = y_probas_forest[:, 1] # score = proba of positive class  
fpr_forest, tpr_forest, thresholds_forest = roc_curve(y_train_5, y_scores_forest)
```

```
plt.plot(fpr, tpr, "b:", label="SGD")  
plot_roc_curve(fpr_forest, tpr_forest, "Random Forest")  
plt.legend(loc="lower right")  
plt.show()  
>>> roc_auc_score(y_train_5, y_scores_forest)  
0.9983436731328145
```

Multiclass Classification

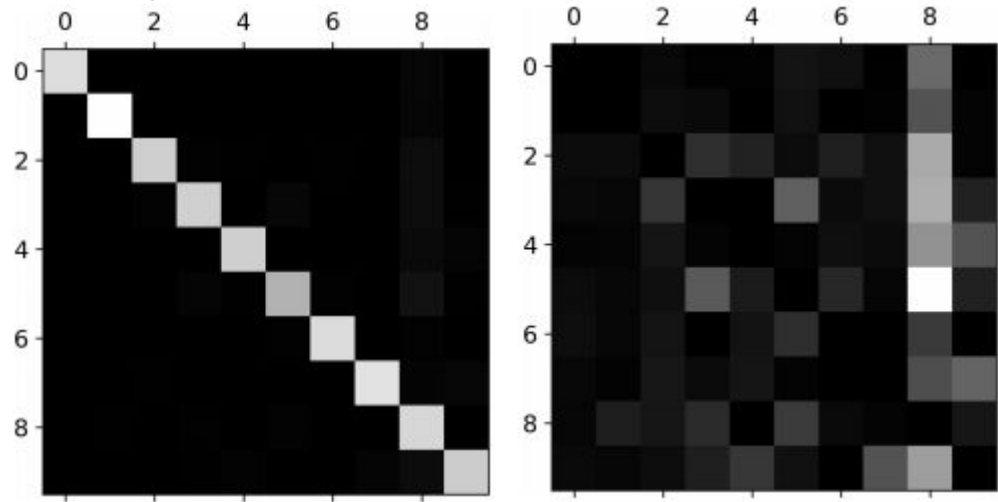
- Binary classification – distinguish between two classes
 - “5” vs “Not 5”
- Multiclass classification (also known as multinomial classification) – distinguish between many classes
- Some classifiers can handle multiple classes by default
 - e.g. SGD, Random Forest, Naïve Bayes
- Others (like Logistic Regression or SVM) are strictly binary classifiers – however, they can be used for multiclass classification in certain strategies
 - One-versus-the-rest/One-vs-all (OVR) – one classifier for each class, highest score classifier taken
 - One-vs-one (OvO) – one classifier for each class pair, $N(N-1)/2$ classifiers to train if there are N classes



Error Analysis

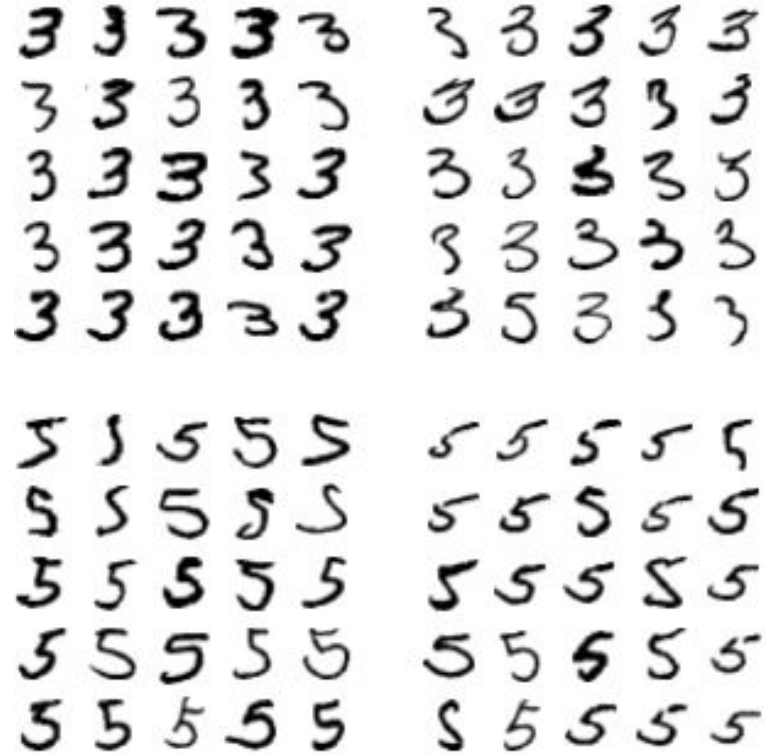
- Look at confusion matrix – easier to understand as image
- Bright diagonal indicates most images were correctly classified
- Many images misclassified as 8 (bright column) – but images with 8 are correctly classified (dark row)

```
>>> y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
>>> conf_mx = confusion_matrix(y_train, y_train_pred)
>>> conf_mx
array([[5578,    0,   22,    7,    8,   45,   35,    5,  222,    1],
       [    0, 6410,   35,   26,    4,   44,    4,    8,  198,   13],
       [  28,   27, 5232,  100,   74,   27,   68,   37,  354,   11],
       [  23,   18,  115, 5254,    2,  209,   26,   38,  373,   73],
       [  11,   14,   45,   12, 5219,   11,   33,   26,  299,  172],
       [  26,   16,   31,  173,   54, 4484,   76,   14,  482,   65],
       [  31,   17,   45,    2,   42,   98, 5556,    3,  123,    1],
       [  20,   10,   53,   27,   50,   13,    3, 5696,  173,  220],
       [  17,   64,   47,   91,    3,  125,   24,   11, 5421,   48],
       [  24,   18,   29,   67,  116,   39,    1,  174,  329, 5152]])
```



3 or 5?

- Bottom-left block are 5s that are incorrectly classified as 3
- Top-right block are 3s incorrectly classified as 5
- What are the issues with the classifier, and why does it struggle to classify these two classes?



Multilabel Classification

- Classification system outputting multiple binary tags
- Measure F1 score for each label, and take average across all labels
- Can weight the labels according to their support – i.e. how many instances of that label are there?



Multiclass Classification

- Each label can be multiclass (i.e. have more than two possible values)
- Input ☐ noisy digit image (array of pixel intensities)
- Output ☐ clean digit image (array of pixel intensities)
- One label per pixel, and each label can have multiple values
- Note: very similar to a regression problem – predicting pixel intensity is more akin to regression

```
noise = np.random.randint(0, 100, (len(X_train), 784))  
X_train_mod = X_train + noise  
noise = np.random.randint(0, 100, (len(X_test), 784))  
X_test_mod = X_test + noise  
y_train_mod = X_train  
y_test_mod = X_test
```



```
knn_clf.fit(X_train_mod, y_train_mod)  
clean_digit = knn_clf.predict([X_test_mod[some_index]])  
plot_digit(clean_digit)
```

